EE1007
Introduction to Computer Science Laboratory

Lab 9
Array

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw

1

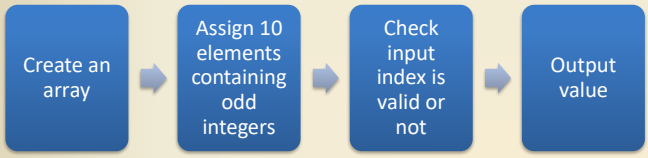
Lab 9.1
P1130295. Parity(30%)

- Test target: array
- You are asked to set an array with 10 elements containing the odd integers from 1 to 19 (i.e., 1, 3, 5, ..., 19). First, enter the index (element). Then, output the corresponding value in the array.

2

Lab 9.1
P1130295. Parity(30%)

◆思考方向



3

Lab 9.1
P1130295. Parity(30%)

◆ Create an array and assign elements

```
const int arraySize = 10;
int s[arraySize];

for (int i = 0; i < arraySize; i++) {
    s[i] = 1 + 2 * i;
}
```

◆ Check input index is valid or not

```
do {
    cout << "Please enter the index of the element (0 to 9): \n";
    cin >> index;

    if (index < 0 || index >= arraySize) {
        cout << "Invalid index. Please enter a value between 0 and 9." << endl;
    }
} while (index < 0 || index >= arraySize);
```


4

Lab 9.2
P113030. Maximum Path Sum(30%)

- Goal: array
- You are asked to write a program to calculate the sum of values within the selected index range.
- First, the user will enter the size of the array, ex: 10. Then, enter the elements of the array, ex. -1.5 -3.5 3.0 -2.4 -1.8 -3.0 4.5 5.7 -8.9 3.4 . Next, enter two integers, representing the selected index range ex.2 5. Finally, output the sum of the selected index range.

5

Lab 9.2
P113030. Maximum Path Sum(30%)

- 思考方向



6

Lab 9.2
P113030. Maximum Path Sum(30%)

- Input valid array size & elements

```
while (true) {
    cout << "Please enter the elements of the array: " << endl;
    bool valid = true;

    for (int i = 0; i < n; ++i) {
        double temp;

        if (!(cin >> temp)) {
            valid = false;
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            break;
        }

        if (temp < -2147483648.0 || temp > 2147483647.0) {
            valid = false;
            break;
        }

        arr[i] = static_cast<long long>(temp * 10 + (temp >= 0 ? 0.5 : -0.5));
    }

    if (!valid) {
        cout << "Wrong Input! Please enter again:" << endl;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        continue;
    }

    break;
}
```


Lab 9.2
P113030. Maximum Path Sum(30%)

- Input valid range

```
while (true) {
    cout << "Please enter two integers, start and end, representing the selected index range: "

    if (!(cin >> start >> end)) {
        cout << "Wrong Input! Please enter again:" << endl;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        continue;
    }

    if (start < 0 || start > end || end >= n) {
        cout << "Wrong Input! Please enter again:" << endl;
        continue;
    }

    break;
}
```

Lab 9.2
P113030. Maximum Path Sum(30%)

- Output the sum of the selected index range

```
long long sum = 0;

for (int i = start; i <= end; ++i) {
    sum += arr[i];
}

double final_sum = sum / 10.0;

cout << fixed << setprecision(1);
cout << "The sum of the values in the index range [" << start << ", " << end << "] is: " << final_sum
```


Lab 9.3
P113031. Lexicographical Ordering (30%)

- Test target: array
- The Lexicographical Algorithm is a method used to generate the next permutation of a sequence based on lexicographical ordering.
- You are asked to write a program to generate all permutations of distinct integers in lexicographical order. Repeat generating and printing each new permutation until the sequence is in descending order.

Lab 9.3
P113031. Lexicographical Ordering (30%)

◆思考方向



Lab 9.3
P113031. Lexicographical Ordering (30%)

◆ Identify pivot and find rightmost successor

```
bool nextPermutation(int arr[], int size) {
    if (size < 2)
        return false;

    int k = -1;

    for (int i = size - 2; i >= 0; i--) {
        if (arr[i] < arr[i + 1]) {
            k = i;
            break;
        }
    }

    if (k == -1)
        return false;

    int l = -1;

    for (int i = size - 1; i > k; i--) {
        if (arr[i] > arr[k]) {
            l = i;
            break;
        }
    }
```

Lab 9.3
P113031. Lexicographical Ordering (30%)

- Swap with pivot

```
void swap(int a, int b, int arr[]) {
    int temp = arr[a];
    arr[a] = arr[b];
    arr[b] = temp;
}
```

- Reverse the suffix

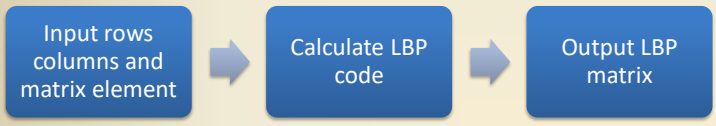
```
void reverse(int arr[], int start, int end) {
    while (start < end) {
        swap(start, end, arr);
        start++;
        end--;
    }
}
```


Lab 9.4
P113032. Local Binary Patterns (20%)

- Goal: array
- The grayscale image data gray_data is processed to calculate the Local Binary Patterns (LBP) by comparing each pixel in a 3x3 neighborhood. The central pixel in each neighborhood serves as the threshold for determining the values of its eight neighbors. If a neighbor's pixel value is greater than or equal to the central pixel, it is assigned a value of 1, otherwise 0.

Lab 9.4
P113032. Local Binary Patterns (20%)

◆思考方向



Lab 9.4
P113032. Local Binary Patterns (20%)

◆ Input rows and columns

```
do {
    inputValid = true;
    std::cout << "Please input the grayscale image matrix (each element should be an integer between 0 and 255):\n";
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cin >> image[i][j];
            if (image[i][j] < 0 || image[i][j] > 255) {
                std::cout << "Invalid input detected! Matrix values must be between 0 and 255.\n";
                inputValid = false;
                break;
            }
        }
        if (!inputValid) break;
    }
} while (!inputValid);
```

Lab 9.4
P113032. Local Binary Patterns (20%)

- Calculate LBP code

```
if (rows >= 3) {
    for (int i = 1; i < rows - 1; ++i) {
        for (int j = 1; j < cols - 1; ++j) {
            std::cout << "LBP Matrix:\n";
            for (int i = 0; i < rows; ++i) {
                for (int j = 0; j < cols; ++j) {
                    std::cout << std::setw(5) << lbp_matrix[i][j];
                }
                std::cout << std::endl;
            }
            lbp_code |= (image[i + 1][j] >= center) << 6;
            lbp_code |= (image[i + 1][j - 1] >= center) << 5;
            lbp_code |= (image[i][j - 1] >= center) << 3;

            lbp_matrix[i][j] = lbp_code;
        }
    }
} else {
    std::cout << "The size of the matrix must be at least 3x3 to compute LBP." << std::endl;
    return 0;
}
```


Lab 9.4
P113032. Local Binary Patterns (20%)

- Output LBP matrix

```
std::cout << "LBP Matrix:\n";
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        std::cout << std::setw(5) << lbp_matrix[i][j];
    }
    std::cout << std::endl;
}
```

Evaluation

- Lab9.1(30%)
  - 5 testcases, 6% each
- Lab9.2(30%)
  - 5 testcases, 6% each
- Lab9.3(30%)
  - 5 testcases, 6% each
- Lab9.4(20%)
  - 5 testcases, 4% each





Andy, Yu-Guang Chen

Assistant Professor, Department of EE, NCU

Email: andyygchen@ee.ncu.edu.tw